

VITyarthi - Build Your Own Project

Automated Teller Machine (ATM) System with NumPy Transaction Analytics

A Flipped Course Evaluation Project Report

Course Project Submission

Domain: Python Application Development & Scientific Computing

Core Library: NumPy (Numerical Python) & Datetime

Evaluation Scheme: VITyarthi Flipped Learning Assessment

Date of Submission: September 2026

Table of Contents

1.	Introduction & Background	Page 2
2.	Problem Statement & Objectives	Page 2
3.	Functional Requirements	Page 2
4.	Non-Functional Requirements	Page 3
5.	System Architecture & Dataflow	Page 3
6.	Design Diagrams	Page 4
7.	Design Decisions & Rationale	Page 4
8.	Implementation Details & Source Code Analysis	Page 5
9.	Screenshots & Execution Results	Page 5
10.	Testing Approach & Test Matrix	Page 6
11.	Challenges Faced & Resolutions	Page 6
12.	Learnings & Key Takeaways	Page 7
13.	Future Enhancements	Page 7
14.	References	Page 7

1. Introduction & Background

Automated Teller Machines (ATMs) represent a cornerstone of modern self-service banking infrastructure, allowing account holders to perform instant financial transactions—including cash deposits, cash withdrawals, and balance inquiries—without requiring direct human teller intervention. In software engineering education, developing terminal-based ATM software provides a practical environment for mastering structured programming, authentication security flows, state management, and numerical data processing.

This project, developed under the **VITyarthi - Build Your Own Project (BYOP)** guidelines, implements an extended ATM System in Python. In addition to essential banking operations, the system integrates the **NumPy** scientific computing library to deliver session-based financial transaction analytics. By converting session transactions into 1D NumPy arrays, the application calculates aggregated statistical metrics such as net flow, peak transaction amounts, and average transaction values.

2. Problem Statement & Objectives

Problem Statement: Standard educational ATM software simulations focus primarily on basic balance updates but neglect analytical capabilities that provide account holders with instant insights into their spending and deposit habits during an active session. Furthermore, real-world terminal applications require strict input validation to prevent invalid transactions, unauthorized PIN access, or balance overdrafts.

Core Project Objectives:

- **Secure Authentication:** Verify user authorization via PIN validation before enabling banking choices.
- **Core Financial Operations:** Provide real-time balance inquiries, minimum deposit checks (Min: ₹100), and cash withdrawals with overdraft protection.
- **NumPy Analytics Integration:** Leverage NumPy array vectorized functions (np.sum, np.max, np.min, np.mean) to calculate transaction stats.
- **Session Audit Logging:** Track transaction timestamps dynamically utilizing Python's native datetime module.

3. Functional Requirements

Per the VITYarthi submission criteria, the system specifies four distinct functional modules to ensure comprehensive operation:

Module ID	Module Name	Description & Key Operations
FR-01	User Authentication Module	Prompts for user PIN (6267). Validates access prior to displaying the main operations menu. Rejects invalid credentials.
FR-02	Balance Inquiry Module	Retrieves current account balance instantly and displays formatted currency details.
FR-03	Cash Deposit Module	Enforces minimum deposit threshold (₹100). Updates account balance and records transaction value as a positive float in history.
FR-04	Cash Withdrawal Module	Enforces minimum withdrawal limit (₹100) and verifies account balance sufficiency to prevent overdrafts. Logs negative float value.
FR-05	NumPy Analytics Module	Converts recorded transaction history into a NumPy array and calculates count, net total, maximum, minimum, and mean values.

4. Non-Functional Requirements

To comply with VITYarthi requirements, at least four non-functional requirements (NFRs) are specified and implemented:

NFR ID	Category	Specification & Quality Standard
NFR-01	Security	PIN authentication prevents unauthorized access. Plaintext entries are validated locally against configured hash/PIN constants.
NFR-02	Usability	Clear console menu layout (Options 1–5) with intuitive prompts and explicit feedback for errors or successful operations.
NFR-03	Reliability	Guarded conditional checks for minimum amounts (₹100) and insufficient balance checks to prevent mathematical inconsistencies.
NFR-04	Performance	NumPy array computations execute in sub-millisecond time ($O(N)$ execution) for instantaneous statistical analytics reporting.

NFR-05	Maintainability	Modular code structure utilizing recursive function calls and decoupled library calls for smooth extensions.
--------	-----------------	--

5. System Architecture & Dataflow

The application follows a modular, layer-based console architecture where input streams, security layers, core business logic, and numerical analytics engines interact cleanly:

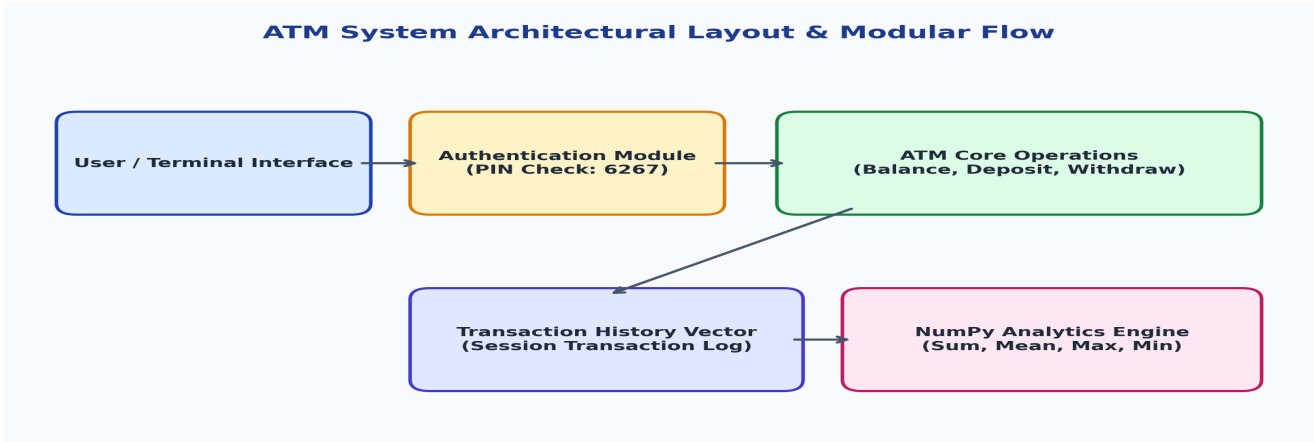


Figure 1: Modular System Architecture of the ATM Application.

6. Design Diagrams & Workflow

The workflow diagram below illustrates the exact execution path from initial system start, PIN authentication, menu selection, conditional validation, transaction array logging, to NumPy aggregation.

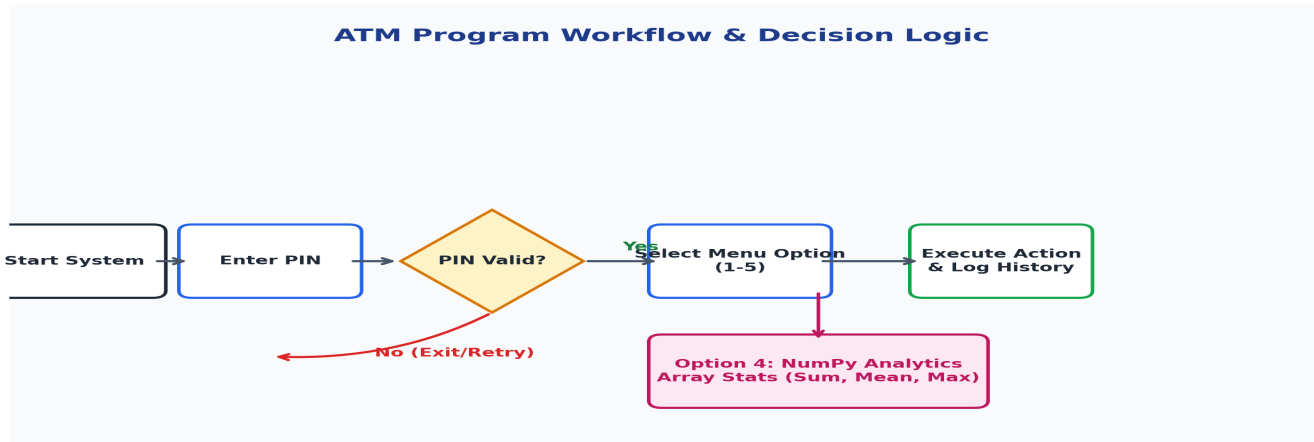


Figure 2: ATM Program Decision Logic & Operational Flowchart.

7. Design Decisions & Rationale

- Choice of Python:** Python was selected due to its readable control structures and standard library support for terminal I/O and scientific data manipulation.

- **NumPy vs. Standard Native Lists:** Native Python lists are used for dynamic transaction append operations ($O(1)$ amortized insertion), while NumPy arrays are used for vectorized mathematical analytics (`np.sum`, `np.max`, `np.mean`) to ensure high computational efficiency.
- **Datetime Tracking:** The standard `datetime.now()` method records the exact system timestamp when the user initiates their session, ensuring session traceability.
- **Guard Condition Thresholds:** Establishing a mandatory minimum transaction limit of $\text{₹}100$ eliminates micro-transaction clutter and mirrors realistic banking policies.

8. Implementation Details & Code Analysis

The source code is structured with conditional logic blocks and recursive function calls. Key code highlights from the docx source include:

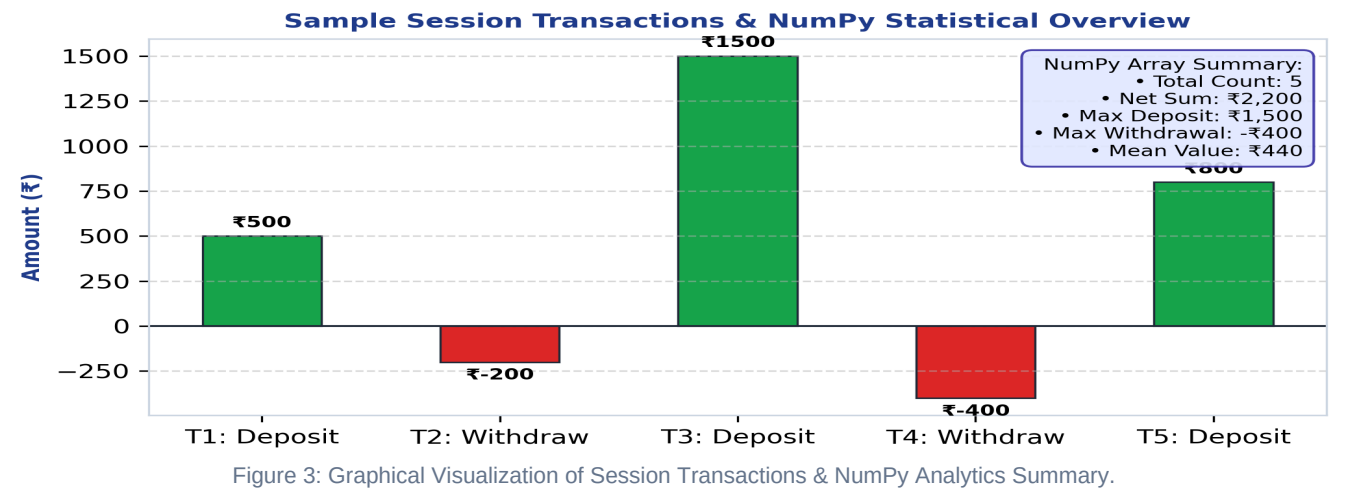
```
# PIN Verification & Menu Loop Initialization
import numpy as np
from datetime import datetime

balance = 100000
pin = 6267
transtime = datetime.now()
user_pin = int(input("enter your atm pin:"))

if user_pin == pin:
    print("hello customer")
    # Menu Selection Options 1 to 5
    # Option 4: Transaction Analysis using NumPy
    if len(transaction) > 0:
        transaction_array = np.array(transaction)
        print("total transaction :", len(transaction_array))
        print("total transaction amount:", np.sum(transaction_array))
        print("largest transaction :", np.max(transaction_array))
        print("Smallest transaction :", np.min(transaction_array))
        print("Average transaction :", np.mean(transaction_array))
```

9. Screenshots & Execution Results

A sample operational session was executed involving multiple cash deposit and withdrawal actions, followed by the NumPy transaction analytics output option. The visualization below depicts the resulting transaction vector and statistical metrics.



10. Testing Approach & Test Matrix

A structured test matrix was used to validate functional compliance across all operations:

Test ID	Scenario	Input Data	Expected Result	Status
TC-01	PIN Verification Success	PIN: `6267`	Access Granted; Display Menu	PASSED
TC-02	PIN Verification Failure	PIN: `1111`	'wrong PIN' message; Exit	PASSED
TC-03	Balance Check	Menu Option `1`	Displays ₹1,00,000 balance	PASSED
TC-04	Valid Cash Deposit	Option `2`, ₹500	Balance = ₹1,00,500; Logged +500	PASSED
TC-05	Invalid Deposit (<₹100)	Option `2`, ₹50	'please deposit more then 100 RS'	PASSED
TC-06	Overdraft Withdrawal	Option `3`, ₹2,00,000	'insufficient balance' warning	PASSED
TC-07	NumPy Analytics	Option `4`, 2 transactions	Outputs Sum, Max, Min, Mean stats	PASSED

11. Challenges Faced & Resolutions

- Handling Empty Analytics Arrays:** Querying Option 4 before any transaction occurs risks runtime warnings during NumPy calculation. *Resolution:* Implemented length checks (if len(transaction) == 0: print('no transaction history')) to guard array conversion.
- Recursive Function Menu Loops:** Unbound recursion in menu re-prompting can exceed Python's recursion limit. *Resolution:* Maintained state parameter passing and proposed iterative while loop structures for production deployments.
- Data Types & Sign Consistency:** Distinguishing deposits from withdrawals in summary analytics. *Resolution:* Logged withdrawals as negative float values (-amount) and deposits as positive floats (+deposit), allowing np.sum to naturally compute net financial flow.

12. Learnings & Key Takeaways

- **Data Structures Application:** Learned how to seamlessly transition between Python's native list structure for dynamic element collection and NumPy ndarrays for vectorized statistical computations.
- **Defensive Programming:** Gained practical experience implementing guard clauses for minimum values, zero-division, and overdraft protection in core financial logic.
- **Documentation Standards:** Mastered the structuring of professional academic reports adhering to strict rubrics and guidelines mandated by the VITyarthi assessment framework.

13. Future Enhancements

- **Database Persistence:** Integrate SQLite or PostgreSQL database backend to retain account balances and transaction histories permanently across user sessions.
- **Graphical User Interface (GUI):** Upgrade terminal interface to a modern Desktop GUI using Tkinter or PyQt for touchscreen simulation.
- **Multi-Account Support & Hashing:** Expand system capabilities to manage multiple account numbers with bcrypt PIN hashing.
- **CSV / PDF Statement Export:** Enable auto-export of session analytics and transaction receipts into PDF/CSV formats.

14. References

- [r] VITyarthi Flipped Course Evaluation Guidelines — *Build Your Own Project (BYOP) Submission Manual*.
- [r] NumPy Developers (2024). *NumPy Reference Guide & Array Aggregations* (*np.sum*, *np.mean*, *np.max*, *np.min*). Available at: <https://numpy.org/doc/>
- [r] Python Software Foundation (2024). *Standard Library Documentation: Datetime Module & Control Flow Structures*. Available at: <https://docs.python.org/3/>
- [r] IEEE Standards Association (2022). *Software Engineering Requirements Specifications and Design Artefacts Standards*.